Contents

Fused causal self-attention with NVIDIA cuTile **1**
What the kernel computes 1
Contract 2
Code map 2
Verified current evidence 2
Run the core validation on the project-local Windows environment 3

Fused causal self-attention with NVIDIA cuTile

Target: Native Windows RTX 4060, compute capability 8.9
Current status: Core objectives in the six-page course assignment complete (5 November 2026).
Final report: six-page PDF · Markdown source

What the kernel computes

Causal scaled dot-product self-attention is

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{D}} + M\right)V,$$

where the mask M excludes keys after the query's position. The fused implementation runs QK scoring, scaling, causal masking, softmax and P×V in a cuTile kernel. It processes Q/K/V tiles and keeps a running maximum, normalisation sum and weighted-value accumulator rather than materialising the full N×N score/probability matrices in global memory.

For a new score tile S_t, the online-softmax update is

$$m' = \max(m, m_t), \quad \alpha = e^{m-m'}, \quad P_t = e^{S_t - m'},$$
$$\ell' = \alpha\ell + \sum_j P_{t,j}, \qquad u' = \alpha u + P_t V_t,$$

and the final result is O=u/ell. Entirely future K/V tiles are skipped; the empty-tile path avoids an undefined -infinity minus -infinity.

Contract

Inputs use FP16 storage and layout Q/K [B,H,N,D], V [B,H,N,Dv]. D and Dv are independently selected from {32,64,128}. The assignment bounds are 1≤B≤16, 1≤H≤32, 1≤N≤8192; Q/K/V are contiguous, self-attention sequence lengths match, and N must be divisible by the selected query and key tile sizes. Outputs are FP16; score, softmax and P×V accumulation are FP32. Partial edge tiles and backward propagation are outside the core contract.

Code map

File	Purpose
src/cutile_attention.py	Fused cuTile kernel and device launch
src/contract.py	Tensor and launch validation
src/reference.py	Independent FP32 NumPy oracle
src/online_reference.py	Tiled online-softmax reference
src/torch_reference.py	CUDA FP32 PyTorch SDPA correctness reference
src/baseline.py	Deliberately unfused FP32 CuPy baseline
run_attention.py	NPZ input / NPY output CLI
tests/	CPU contract and native GPU correctness tests
benchmarks/, scripts/	Timings, numerical reports, NCU and verification

Verified current evidence

- Native RTX 4060: genuine cuTile launch, **57 native cuTile correctness tests passed**, 3 selected optional tests passed, **29/29** numerical configs passed and CLI/NumPy check passed: run #42075683189.
- Trusted FP32 PyTorch SDPA vs NumPy: **7/7 workloads pass**, maximum absolute discrepancy 0.00048828125. Efficient SDPA: 10 configurations × 3 sessions, **30/30 paired correctness checks**; profiler backend aten::_efficient_attention_forward: run #41511770773.
- Benchmarks: seven workload families, 29 legal tile rows, three sessions, 10 warmups, 30 CUDA-event samples and five launches per sample. Thirteen rows satisfy both ≤5% stability

filters. The formal speedup baseline is the unfused FP32 CuPy implementation, not optimized SDPA.

- NCU: 27 native reports, full per-profile metric analysis. Week 2 medium D=128 / 64×64 LOAD_LATENCY=10 hint is workload-specific. Week 3 longer-batched D=128 / 64×128 is 1.60% faster than 64×64 in the stable three-session matrix and 0.68% faster in matched NCU duration, but uses more shared memory/registers without increasing occupancy; keep global tile 64×64 and global hint 0. See Week 3 NCU note.
- Full SASS/PTX pane movement proof: 216 checks, zero failures across 27 reports, 486 before/scrolled/restored captures.

Run the core validation on the project-local Windows environment

```
$py = ".\.venv-cutile-windows\Scripts\python.exe"
& $py scripts\cutile_smoke.py
$env:REQUIRE_CUTILE = "1"
& $py -m pytest -v -ra tests\test_gpu_correctness.py tests\test
& $py scripts\mentor_numerical_matrix.py --output .tmp\validati
& $py run_attention.py --input INPUT.npz --output OUTPUT.npy --
& $py scripts\verify_cli_output.py --input INPUT.npz --output O
```

For complete benchmark methodology, NCU interpretation, performance limitations, optional extensions and the assignment audit, see the final report and course completion checklist.